Operating System Lab Assignment 2

Assignment 2(a):-

Write a C program using signal() system call to handle the reception of SIGINT signal which allows to execute a designated (user) function. Here the function is responsible for creating a child process by using fork() system call and then you have to display the PROCESS ID and PARENT PROCESS ID from the parent process as well as from the child process.

Ans:

```c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <sys/types.h>

void signal_handler(int sig) {
    pid_t pid;
    printf("\nSIGINT received (Signal %d). Handling via user function [cite: 32, 40]\n", sig);

    /* fork() duplicates the process to create a child [cite: 35] */
    pid = fork();

    if (pid < 0) {
        perror("Fork failed");
    } else if (pid == 0) {
        /* Child process displays its PID and parent's PID [cite: 35] */
        printf("Child Process: PID = %d, PPID = %d\n", getpid(), getppid());
        exit(0); /* Terminate child to prevent infinite signal loops [cite: 36] */
    } else {
        /* Parent process displays its PID and the child it created [cite: 35] */
        printf("Parent Process: PID = %d, PPID = %d\n", getpid(), getppid());
    }
}

int main() {
    /* signal() redefines how the process reacts to Ctrl+C [cite: 34, 40] */
    signal(SIGINT, signal_handler);
    printf("PID: %d. You have 15 seconds to press Ctrl+C...\n", getpid());

    /* Use a finite loop to avoid 'Time Limit Exceeded' on web terminals */
    for(int i = 0; i < 15; i++) {
        sleep(1);
    }
    return 0;
}
```

Output:

```
PID: 3. You have 15 seconds to press Ctrl+C...

Time limit exceeded
```

Assignment 2(b):-

Write a C program which will take the Process ID and signal ID as input to demonstrate the use of kill() system call.

Ans:

```c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>

int main() {
    int target_pid, sig_id;

    /* Input PID and Signal ID as required by the assignment  */
    printf("Enter target Process ID: ");
    scanf("%d", &target_pid);
    printf("Enter Signal ID (e.g., 2 for SIGINT, 9 for SIGKILL)[cite: 47, 49]: ");
    scanf("%d", &sig_id);

    /* kill() sends a specific signal to the target PID [cite: 43, 51] */
    if (kill(target_pid, sig_id) == 0) {
        printf("Signal %d successfully sent to PID %d [cite: 50]\n", sig_id, target_pid);
    } else {
        perror("Kill system call failed");
    }

    return 0;
}
```

```
Enter target Process ID: 123
Enter Signal ID (e.g., 2 for SIGINT, 9 for SIGKILL)[cite: 47, 49]: 47
Kill system call failed: No such process
```

Assignment 2(c):-

Write a C program to create a user level thread using system call pthread_create( ) and assign the thread to display the “HELLO WORLD” . Use pthread_exit() in your program (if possible) for terminating the thread.

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h> /* Required for thread system calls  */

/* Function executed by the thread to display HELLO WORLD  */
void* thread_work(void* arg) {
    printf("HELLO WORLD\n");
    /* pthread_exit() terminates the thread specifically [cite: 54, 57] */
    pthread_exit(NULL);
}

int main() {
    pthread_t my_thread;

    /* pthread_create() spawns the thread and assigns the function [cite: 53, 57] */
    if (pthread_create(&my_thread, NULL, thread_work, NULL) != 0) {
        perror("Thread creation failed");
        return 1;
    }

    /* Wait for thread to finish before main process exits [cite: 55] */
    pthread_join(my_thread, NULL);

    return 0;
}
```

```
HELLO WORLD
```